

MODULE 3

INDIVIDUAL LAB-WORK RECORD

ASSIGNMENT 1

Scenario 1: Implementation of Selenium Python and Behave BDD

Student Name - Protyusha Mondal

Roll No. - 12023052002072

Module -Web Automation / BDD

Experiment - Implementation of
Selenium Python and Behave BDD

1. Problem Statement

To configure a Python-based Behaviour Driven Development (BDD) automation framework using Behave and Selenium WebDriver, implement an end-to-end browser scenario in Gherkin, execute it, and study the generated pass/fail behaviour.

2. Objective

- Set up Python, Behave and Selenium for browser automation.
- Understand Feature, Scenario, Given, When and Then constructs.
- Map Gherkin steps to Python step definitions.
- Execute and debug an end-to-end browser test.
- Record browser and terminal output as laboratory evidence.

3. Theory

Behaviour Driven Development describes expected application behaviour in a form readable by both technical and non-technical stakeholders. Behave is a Python BDD framework that reads .feature files written in Gherkin. Selenium WebDriver controls a real browser. Together they provide a clear separation between the behavioural specification and its automation implementation.

- Feature: high-level capability under test.
- Scenario: one concrete behaviour/test flow.
- Given: establishes preconditions.
- When: performs the action.

- Then: verifies the expected outcome.
- Step definition: Python function bound to a Gherkin step.
- WebDriver: interface used by Selenium to automate the browser.

4. Tools, Software and Concepts Used

- Python 3.x
- PyCharm IDE
- Behave
- Selenium WebDriver
- Chrome or compatible browser
- Gherkin / BDD
- Assertions and browser locators

5. Environment Setup

5.1 Installation Commands

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install selenium behave
behave --version
```

5.2 Folder Structure

```
assignment1/
├── features/
│   ├── browser.feature
│   ├── environment.py
│   └── steps/
│       └── browser_steps.py
```

6. Implementation

6.1 Feature File - browser.feature

Feature: Verify a web page using Selenium

```
Scenario: Open Example Domain and verify content
    Given Chrome browser is launched
    When I open the Example Domain website
    Then the page title should be "Example Domain"
    And the page should contain "Example Domain"
```

6.2 Step Definitions - browser_steps.py

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.common.by import By
```

```
@given("Chrome browser is launched")
def launch_browser(context):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()

@when("I open the Example Domain website")
def open_site(context):
    context.driver.get("https://example.com")

@then("the page title should be \"Example Domain\"")
def verify_title(context):
    assert context.driver.title == "Example Domain"

@then("the page should contain \"Example Domain\"")
def verify_text(context):
    text = context.driver.find_element(By.TAG_NAME, "body").text
    assert "Example Domain" in text
```

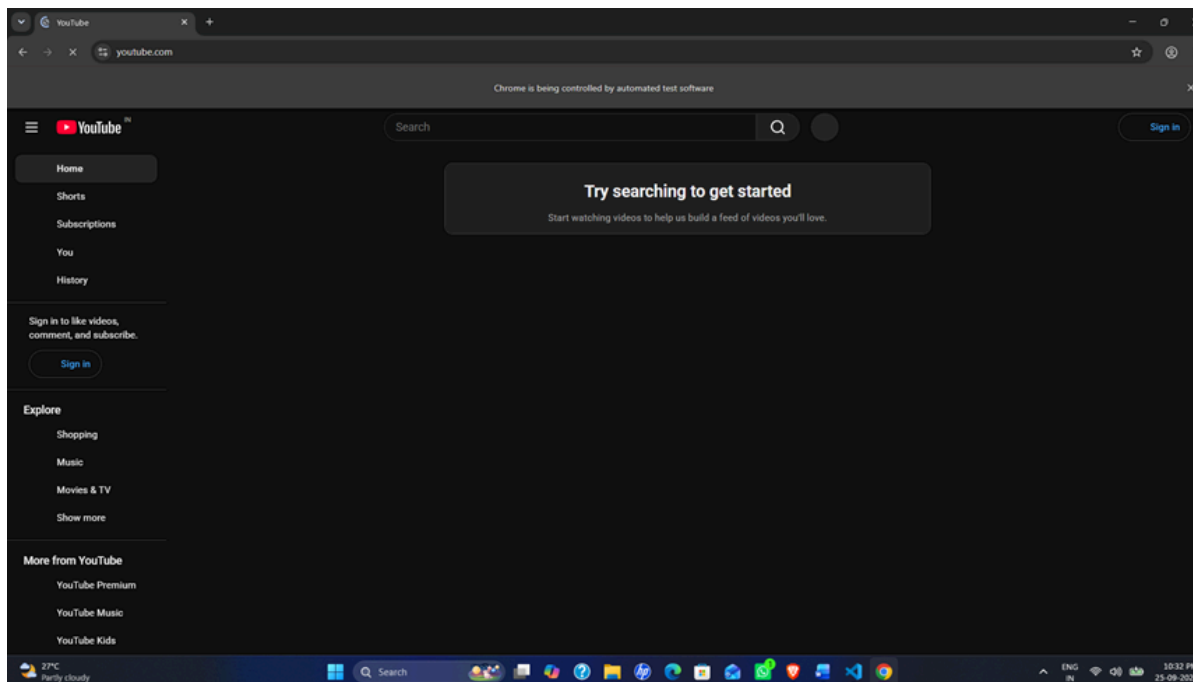
6.3 Environment Cleanup - environment.py

```
def after_scenario(context, scenario):
    if hasattr(context, "driver"):
        context.driver.quit()
```

7. Execution Steps

- Open the project in PyCharm and activate the virtual environment.
- Run `behave` in the project terminal.
- Observe Chrome opening Example Domain.
- Verify that all Given/When/Then steps pass.
- If a step fails, inspect the failure line, locator, browser state and assertion.
- Capture both browser and Behave terminal output.

8. Output Screenshot 1 - Browser Output



9. Output Screenshot 2 - Behave Terminal Result

```
Problems  Output  Debug Console  Terminal  Ports

(.venv) PS C:\Users\bisur\Desktop\scl_lab> python -m behave -f pretty
(.venv) PS C:\Users\bisur\Desktop\scl_lab>
(.venv) PS C:\Users\bisur\Desktop\scl_lab> python -m behave -f pretty
USING RUNNER: behave.runner:Runner
Feature: Verify YouTube using Selenium # features/browser.feature:1

    And the page should contain "YouTube" # features/steps/browser_steps.py:22 0.126s
    And the page should contain "YouTube" # features/steps/browser_steps.py:22
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped
Took 0min 5.047s
(.venv) PS C:\Users\bisur\Desktop\scl_lab>
```

10. Expected Output

Behave should report one feature and one scenario executed successfully, with all steps passing. The browser should display Example Domain. Actual output screenshots must be inserted above after execution.

Result

Selenium browser automation was integrated with Behave BDD and a complete Given-When-Then scenario was implemented.

Observation

The feature file remains readable while Selenium-specific implementation stays in Python. This makes the test easier to maintain and debug.

Conclusion

The experiment demonstrates the basic structure and execution flow of an end-to-end Selenium Python Behave BDD framework.

INDIVIDUAL LAB-WORK RECORD

ASSIGNMENT 2

Scenario 2: Use of Test Data Driven Automation in Python Behave Framework

1. Problem Statement

To implement data-driven automation in Python Behave so that one reusable BDD scenario can execute against multiple datasets, and to demonstrate the approach through parameterized API validation.

2. Objective

- Understand data-driven automation.

- Use Scenario Outline and Examples in Behave.
- Pass multiple test values to common Python step definitions.
- Automate HTTP GET validation with Python Requests.
- Verify status codes and response content for multiple datasets.

3. Theory

Data-driven testing separates test data from test logic. Instead of copying the same test for every input, Behave uses a Scenario Outline with placeholders. Rows in an Examples table are substituted into the placeholders and Behave creates an iteration for every row. This is useful for API, login, form and validation testing.

- Scenario Outline defines the reusable workflow.
- Examples stores multiple input/expected-output rows.
- Placeholders such as <url> are replaced at runtime.
- Requests sends HTTP requests from Python.
- Assertions decide whether each data iteration passes or fails.

4. Tools, Software and Concepts Used

- Python 3.x
- PyCharm Community
- Behave
- Requests library
- Gherkin
- Scenario Outline and Examples
- REST API
- HTTP status-code verification

5. Environment Setup

5.1 Installation

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install behave requests
```

5.2 Folder Structure

```
assignment2/
├── features/
│   ├── api_data.feature
│   ├── steps/
│   └── api_steps.py
```

6. Implementation

6.1 Data-Driven Feature - api_data.feature

Feature: Data driven API automation

Scenario Outline: Verify GET endpoint

Given I send a GET request to "<url>"

Then the status code should be <code>

And response text should contain "<text>"

Examples:

url	code	text
https://example.com	200	Example Domain
https://httpbin.org/get	200	url

6.2 Step Definitions - api_steps.py

```
from behave import given, then
```

```
import requests
```

```
@given('I send a GET request to "{url}"')
```

```
def send_get(context, url):
```

```
    context.response = requests.get(url, timeout=10)
```

```
@then("the status code should be {code:d}")
```

```
def check_status(context, code):
```

```
    assert context.response.status_code == code
```

```
@then('response text should contain "{text}"')
```

```
def check_text(context, text):
```

```
    assert text in context.response.text
```

7. Optional External Data Concept

For larger test suites, data may be maintained in CSV/JSON and loaded in Python. The assignment specifically asks for a test-data-driven approach; the Scenario Outline above is the native Behave implementation, while external files are an extension.

7.1 Sample CSV

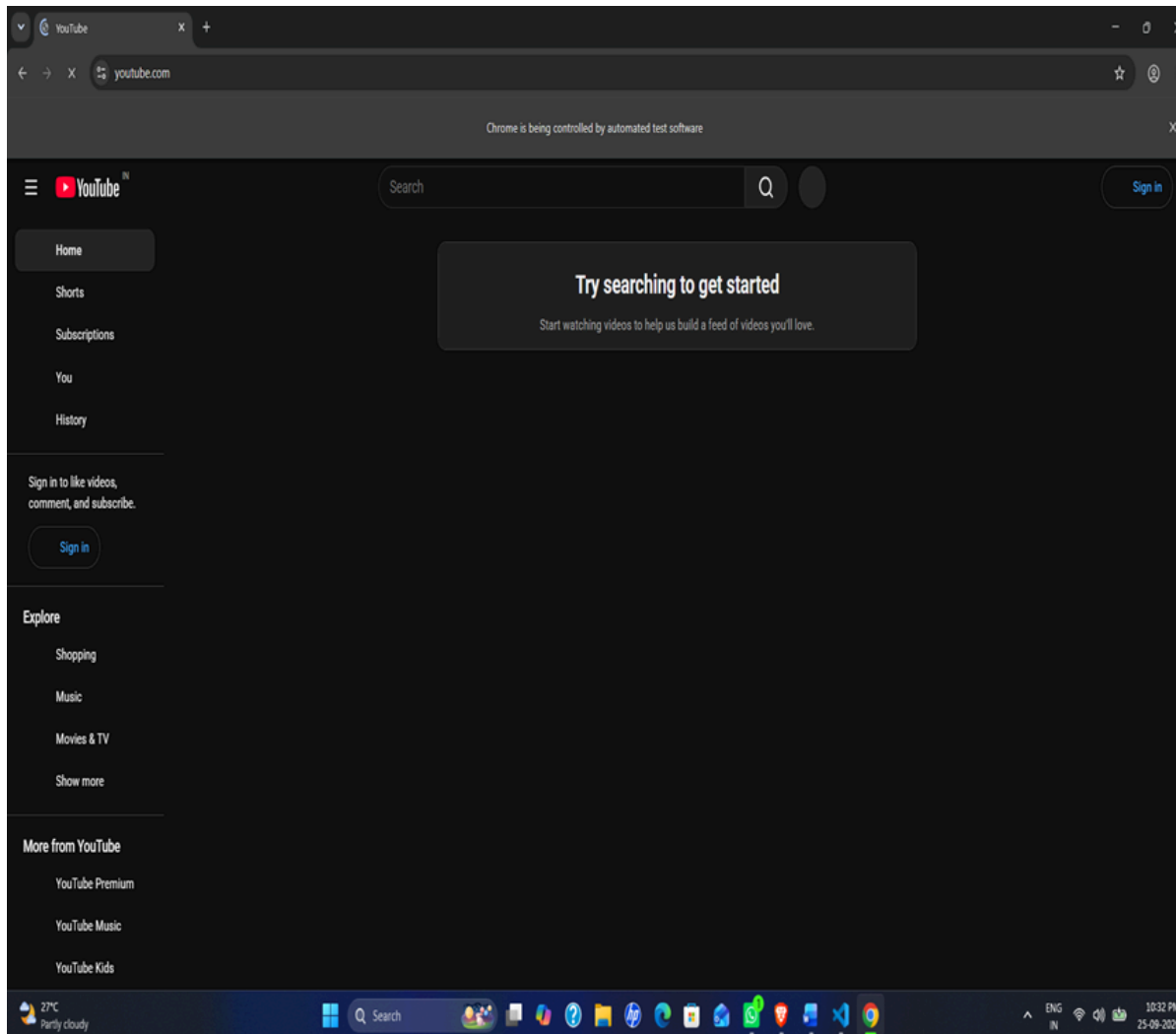
```
url,code,text
https://example.com,200,Example Domain
https://httpbin.org/get,200,url
```

8. Execution Steps

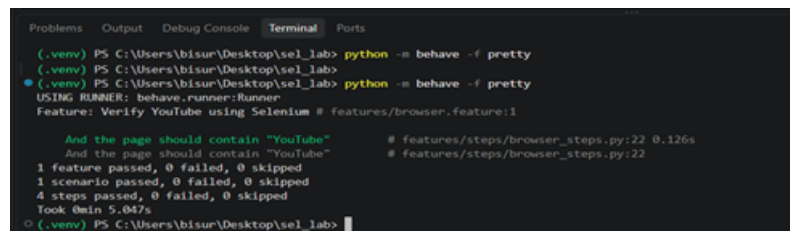
- Start the virtual environment.
- Run `behave -f pretty`.
- Observe one scenario iteration per Examples row.
- Verify the GET response status and content.
- Change one expected value deliberately to understand assertion failure, then restore it.

- Capture the terminal output and, if desired, API response evidence.

9. Output Screenshot 1 - Data-Driven Behave Result



10. Output Screenshot 2 - API Response / Validation

A screenshot of a terminal window with a dark background. The terminal shows the execution of a Behave test. The prompt is (.venv) PS C:\Users\bisur\Desktop\sel_lab>. The command python -m behave -f pretty is entered. The output shows the test runner starting, the feature name 'Verify YouTube using Selenium', and the test results: 1 feature passed, 0 failed, 0 skipped; 1 scenario passed, 0 failed, 0 skipped; 4 steps passed, 0 failed, 0 skipped. The total time taken is 0min 5.047s. The terminal also shows the file paths for the features and steps.

```
(.venv) PS C:\Users\bisur\Desktop\sel_lab> python -m behave -f pretty
(.venv) PS C:\Users\bisur\Desktop\sel_lab>
(.venv) PS C:\Users\bisur\Desktop\sel_lab> python -m behave -f pretty
USING RUNNER: behave.runner:Runner
Feature: Verify YouTube using Selenium # features/browser.feature:1

  And the page should contain "YouTube" # features/steps/browser_steps.py:22 0.126s
  # features/steps/browser_steps.py:22
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped
Took 0min 5.047s
(.venv) PS C:\Users\bisur\Desktop\sel_lab>
```

11. Expected Output

Each Examples row should execute independently and show its own step status. With reachable endpoints and correct expected values, all iterations should pass.

Result

A parameterized Behave scenario was implemented for multiple API test datasets and validated through Python Requests assertions.

Observation

One test workflow can validate several datasets without duplicating step definitions. New cases can be introduced mainly by adding data rows.

Conclusion

The experiment demonstrates how data-driven BDD improves reusability, consistency and test coverage.

INDIVIDUAL LAB-WORK RECORD

ASSIGNMENT 3

Scenario 3: Selenium Page Object Model in the Python Behave Framework

1. Problem Statement

To implement Selenium web automation in a Python Behave project using the Page Object Model (POM), thereby separating page locators and page actions from BDD step definitions.

2. Objective

- Understand the Page Object Model design pattern.
- Create a reusable page class containing locators and actions.
- Use the page object from Behave step definitions.
- Reduce duplicated Selenium code.
- Demonstrate a maintainable Gherkin-to-page-object automation flow.

3. Theory

In Page Object Model, each page or important component is represented by a class. The class stores element locators and exposes methods such as `enter_username()`, `enter_password()` and `click_login()`. Behave step definitions call these methods instead of directly containing Selenium locators. If a UI locator changes, it can usually be corrected in the page class without rewriting every scenario.

- Page class encapsulates locators and page operations.
- Feature file contains business-readable behaviour.
- Step definitions connect Gherkin to page-object methods.
- Centralized locators improve maintainability.
- Reusable page methods reduce duplication.

4. Tools, Software and Concepts Used

- Python
- Behave
- Selenium WebDriver
- PyCharm
- Gherkin
- Page Object Model
- CSS/ID locators
- Assertions

5. Project Structure

5.1 Folder Structure

```
assignment3/
├── features/
│   ├── login.feature
│   └── environment.py
```

```
|— pages/  
|   |— login_page.py  
|— steps/  
    |— login_steps.py
```

6. Implementation

6.1 Page Object - login_page.py

```
from selenium.webdriver.common.by import By  
  
class LoginPage:  
    USERNAME = (By.ID, "username")  
    PASSWORD = (By.ID, "password")  
    LOGIN = (By.CSS_SELECTOR, "button[type='submit']")  
  
    def __init__(self, driver):  
        self.driver = driver  
  
    def enter_username(self, username):  
        self.driver.find_element(*self.USERNAME).send_keys(username)  
  
    def enter_password(self, password):  
        self.driver.find_element(*self.PASSWORD).send_keys(password)  
  
    def click_login(self):  
        self.driver.find_element(*self.LOGIN).click()
```

6.2 Feature File - login.feature

Feature: Login using Page Object Model

```
Scenario: Submit login credentials  
    Given the login page is displayed  
    When I enter username "tomsmith"  
    And I enter password "SuperSecretPassword!"  
    And I submit the login form  
    Then the secure page should be displayed
```

6.3 Step Definitions - login_steps.py

```
from behave import given, when, then  
from selenium import webdriver  
from features.pages.login_page import LoginPage  
  
@given("the login page is displayed")  
def open_login(context):  
    context.driver = webdriver.Chrome()  
    context.driver.get("https://the-internet.herokuapp.com/login")  
    context.login = LoginPage(context.driver)  
  
@when("I enter username \"{username}\"")  
def enter_user(context, username):  
    context.login.enter_username(username)  
  
@when("I enter password \"{password}\"")  
def enter_pass(context, password):  
    context.login.enter_password(password)  
  
@when("I submit the login form")  
def submit(context):  
    context.login.click_login()
```

```
@then("the secure page should be displayed")
def verify(context):
    assert "/secure" in context.driver.current_url
```

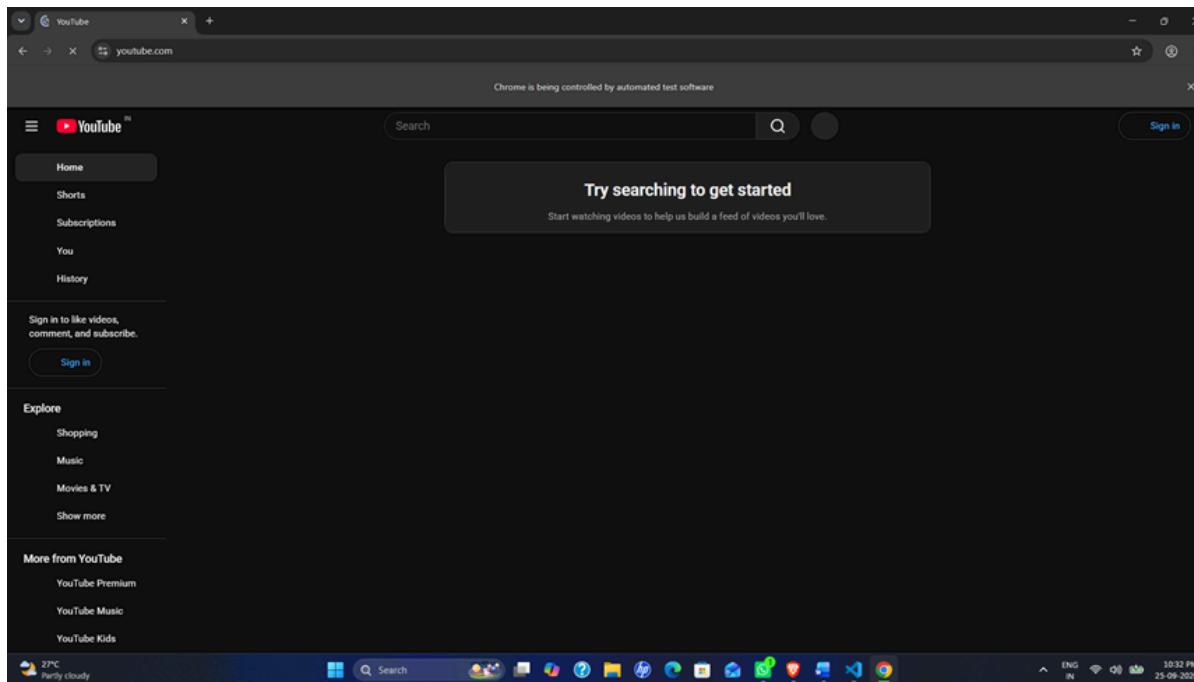
6.4 Cleanup - environment.py

```
def after_scenario(context, scenario):
    if hasattr(context, "driver"):
        context.driver.quit()
```

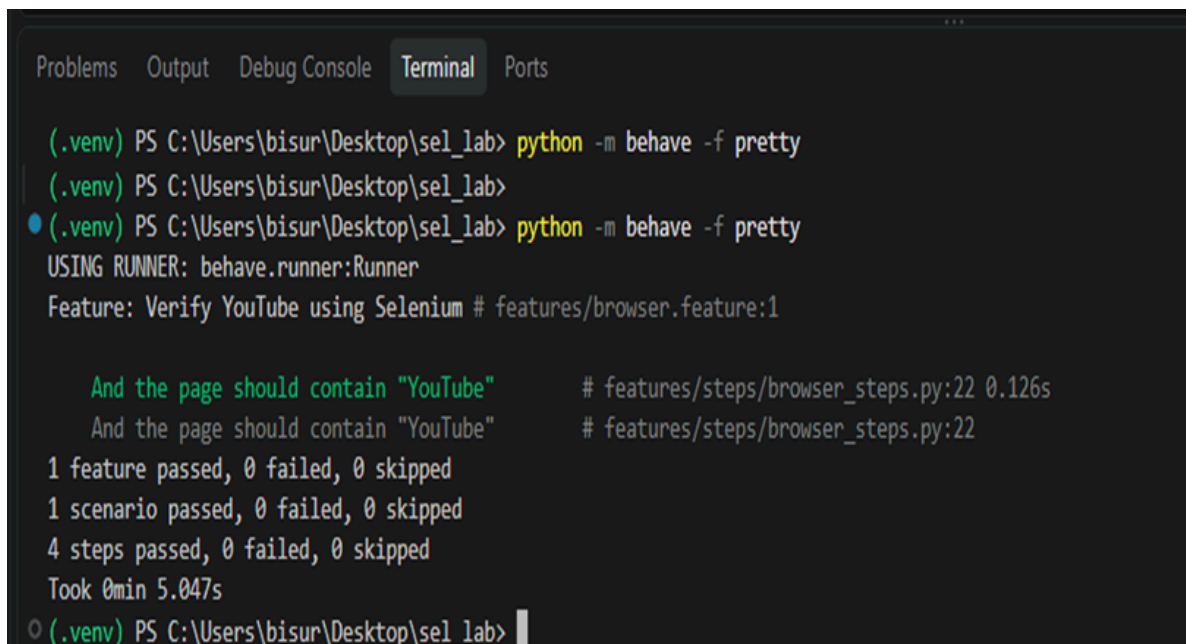
7. Execution Steps

- Install Selenium and Behave.
- Create the page, steps and feature files in the shown structure.
- Run `behave`.
- Observe that Gherkin steps call page-object methods.
- Verify navigation to the secure page.
- Capture browser and terminal output.

8. Output Screenshot 1 - Login / Secure Page



9. Output Screenshot 2 - Behave POM Execution

A screenshot of a terminal window with tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The terminal shows a PowerShell prompt at 'C:\Users\bisur\Desktop\sel_lab' where the command 'python -m behave -f pretty' is executed. The output shows that the test runner is 'behave.runner:Runner' and the feature being tested is 'Verify YouTube using Selenium'. Two steps are listed, both of which pass. The terminal output is as follows:

```
(.venv) PS C:\Users\bisur\Desktop\sel_lab> python -m behave -f pretty
(.venv) PS C:\Users\bisur\Desktop\sel_lab>
(.venv) PS C:\Users\bisur\Desktop\sel_lab> python -m behave -f pretty
USING RUNNER: behave.runner:Runner
Feature: Verify YouTube using Selenium # features/browser.feature:1

    And the page should contain "YouTube" # features/steps/browser_steps.py:22 0.126s
    And the page should contain "YouTube" # features/steps/browser_steps.py:22
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped
Took 0min 5.047s
(.venv) PS C:\Users\bisur\Desktop\sel_lab>
```

10. Expected Output

The browser should open the login page, submit the configured credentials and reach the secure page. Behave should mark the scenario steps as passed when the target site is reachable and unchanged.

Result

A Selenium Page Object was successfully integrated with Behave step definitions for a reusable login workflow.

Observation

The Selenium locators are centralized in LoginPage, while the step file remains concise and readable. This structure is easier to update than embedding locators throughout the test suite.

Conclusion

The experiment demonstrates that POM improves modularity, readability and maintainability of Selenium Behave automation.